

Accelerating Microservice Network Stacks on Kubernetes via a Unified DSL

Hoku Ishibe
Toyohashi University of Technology
Aichi, Japan
hoku@perf.cs.tut.ac.jp

Siyi Hu
Toyohashi University of Technology
Aichi, Japan
husiyi@perf.cs.tut.ac.jp

Ajay Brahmakshatriya
MIT CSAIL
Cambridge, USA
ajaybr@mit.edu

Saman Amarasinghe
MIT CSAIL
Cambridge, USA
saman@csail.mit.edu

Yukinori Sato
Toyohashi University of Technology
Aichi, Japan
yukinori@cs.tut.ac.jp

Abstract—Kubernetes has evolved as the de facto cloud operating system for microservice development. Despite its widespread adoption, two issues remain. First, managing numerous configuration files (e.g., Dockerfiles, manifests) for each microservice reduces developer productivity. Second, inter-container communication constitutes a large fraction of processing time, often becoming a performance bottleneck. To address these issues, we have developed a unified DSL for automatically generating optimized microservices. In this paper, we extend a DSL for a custom protocol stack generation called NetBlocks into a unified DSL for Kubernetes environment generation to accelerate inter-container communication. Our evaluation on a tiny application demonstrates that this approach achieves a 330 times speedup compared to the original gRPC-based implementation.

Index Terms—Domain-Specific Language, Kubernetes, cloud computing, network stack

I. INTRODUCTION

In recent years, Kubernetes has evolved to become the de facto cloud operating system [1]. The 2023 Cloud Native Computing Foundation (CNCF) annual survey shows that 71% of organizations are using Kubernetes in production and 18% are evaluating it [2]. Now, Kubernetes acts as the operating system in modern cloud infrastructures with highly containerized environments. Despite widespread adoption of Kubernetes, two issues remain in application development on Kubernetes: developer productivity in the development phase and inter-container communication latency at runtime.

The first issue is that developing applications for Kubernetes requires writing a large number of configuration and deployment files to specify the behaviors of underlying resources, which results in decreased developer productivity. Here, the key feature of Kubernetes is orchestrating numerous containers, which are called pods in Kubernetes. In typical Kubernetes environments, microservice architecture is assumed, where application business logic is divided into microservices to fit nicely into the underlying software stacks and resources, and each microservice is deployed to a single container (pod).

Although Kubernetes orchestration abstracts the underlying microservices structure, it introduces development complexity

through managing a large number of configuration files, including Kubernetes configurations, container definitions, and inter-container communication. Typically, each microservice requires at least a Kubernetes manifest file and a Dockerfile for each container in addition to microservice logic part. Furthermore, communication interface files such as Protocol Buffer definition in gRPC [3] is usually required for each inter-container communication. When we assume an application with 40 unique microservices, we need to prepare 160 configuration files in a typical case. Hand-coding these configuration files significantly reduces developer productivity, and this process is expected to be automated.

The second issue is that inter-container communication constitutes a large fraction of processing time, often becoming a performance bottleneck. Unlike monolithic architectures where all execution is processed in a single container, microservice architectures incur inter-container communication to process some of the procedural calls that use functions or libraries running in remote containers. This introduces the additional network communication latency, which is often referred to datacenter tax.

To mitigate this, gRPC [3] is widely adopted as a high-performance inter-container communication interface in Kubernetes environments, and now 33% of organizations use it in production services [2]. gRPC uses Protocol Buffers as the interface definition language and provides features such as cross-platform compatibility, efficient binary serialization, bidirectional streaming, and built-in support for authentication, tracing, and load balancing. However, as demonstrated in [4], gRPC overhead in actual application services accounts for a large portion of end-to-end latency due to the complex protocol handling spanning multiple layers, such as TCP/IP and Protocol Buffers.

To address these two issues, we attempt to improve inter-communication latency by replacing gRPC with a minimum custom protocol generated for the target applications. Here, we focus on NetBlocks, which is a DSL for generating custom protocol stacks optimized for specific communica-

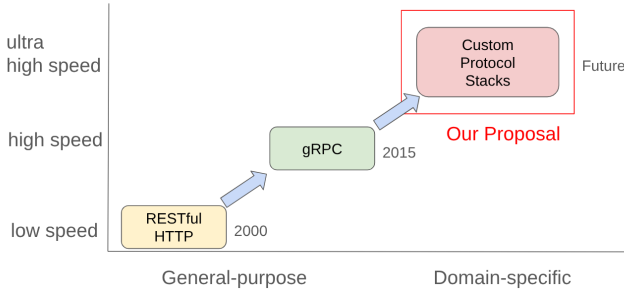


Figure 1: Evolution of communication protocols for microservices

tion patterns [5]. Furthermore, we develop a new unified DSL to automatically generate all of the configuration files for deploying microservices into Kubernetes environments. This unified DSL enables developers to build and deploy microservice applications by writing only application logic in a monolithic style, without manually creating configuration files or implementing and tuning communication code. We believe our DSL-based approach can simultaneously solve the two issues discussed above: automated code generation improves developer productivity, while custom protocol stacks reduce application latency.

In this paper, we implement a communication interface generator that can replace gRPC interface configuration files with those for a custom protocol generated by NetBlocks. Furthermore, we extend the features of NetBlocks as a unified DSL for configuring and managing Kubernetes environments. We evaluate our approach by measuring developer productivity when implementing microservice applications using the unified DSL, and runtime performance when using the unified DSL with NetBlocks-generated protocol stacks compared to the original unified DSL implementation using gRPC. Our evaluation demonstrates 71 times improvement in developer productivity and 330 times speedup in application performance.

II. DSLs FOR CLOUD COMPUTING ENVIRONMENT

Prior works [6] [7] [8] propose cutting-edge DSLs for each scenarios in cloud computing environment and they demonstrates DSLs as key enablers for managing the complexity of cloud computing environment. However, two practical issues remain unaddressed in these works: the overhead of managing a large number of configuration files and the communication overhead of gRPC. We solve both issues by generating application-specific custom protocol stacks integrated into a unified DSL for Kubernetes.

Figure 1 shows the evolution of communication protocols for microservices. REST (Representational State Transfer) [9] was introduced in 2000 and soon became the dominant approach for inter-service communication interface. Fifteen years later, Google developed gRPC to address REST’s performance limitations. The gRPC framework achieves lower latency than RESTful HTTP through HTTP/2 and Protocol Buffers’ binary

serialization. However, gRPC still suffers from the overheads incurred by HTTP/2 and TCP/IP. Further performance improvements are achievable by generating application-specific custom protocol stacks that eliminate unnecessary features from these layers.

In this paper, we use NetBlocks, which is a DSL for a high-performance light-weight protocol stack generator [5], and attempt to apply custom protocol stacks specialized for the communication patterns of target microservice applications. Here, NetBlocks can achieve lower latency compared to general-purpose protocols such as gRPC by eliminating redundant protocol fields found in standard protocol stacks. By reducing the bit length of each field enough to send/receive all the data defined by applications, NetBlocks can prevent unnecessary communication overhead. Furthermore, NetBlocks’ modular and extensible architecture enables developers to customize binary layouts in packets by composing reusable modules, without compiler knowledge. This feature contributes to extending it into a unified DSL for Kubernetes.

NetBlocks is implemented as one of the applications of BuildIt. BuildIt [10] is a framework for developing DSLs using type-based multi-stage programming. NetBlocks leverages BuildIt’s partial evaluation capabilities to automatically generate optimized protocol implementations. BuildIt supports a principle of the Futamura Projection [11], which is an example of partial evaluation in terms of staged specialization. However, while the Futamura Projection automatically specializes the partial evaluator itself, BuildIt employs explicit programmer-directed staging, where compile-time (static) and runtime (dynamic) computations are distinguished via type annotations, without requiring self-application. As a result, BuildIt’s staging mechanism enables automatic specialization instead of manually implementing and optimizing network stacks for each protocol variation.

However, there is no explicit interface for NetBlocks to integrate it to inter-container communication on Kubernetes environment. So, we propose a unified DSL to bridge the semantic and abstraction gap between NetBlocks and Kubernetes configuration. The unified DSL enables developers to build and deploy microservice applications from a single application logic source file written in a Python-like syntax. By inputting the unified DSL, our proposed framework composed of a DSL compiler and configuration generators automatically generates configuration files such as Kubernetes manifests, Dockerfile and gRPC configurations. As a result of our framework, all developers need to prepare is application logic code with some directive specification for Kubernetes, and we do not have to consider how to write inter-container communication code and Kubernetes-specific settings.

Figure 2 shows an example of directive specification implemented as decorators in Python. Here, we specify the functions to be deployed as microservices and their inter-container communication interface (port numbers). Feeding the input source file into our proposed framework including a DSL compiler and configuration generators automatically generates all necessary files for the application deployment, as

```

@msvc(port=8081)
def caller(a: int) → int:
    b = callee(a)
    c = callee2(b)
    return c

@msvc(port=8082)
def callee(b: int) → int:
    return b + 2

@msvc(port=8083)
def callee2(c: int) → int:
    return c * 2

@msvc(port=8080)
def main() → int:
    return caller(2)

```

Figure 2: Developers write a single file in a Python-like syntax using decorators for microservice configuration and function calls for communication.

```

caller
├── CMakeLists.txt
├── Dockerfile
├── build.sh
├── caller_server_logic.codon
├── caller_server.cc
├── caller_server_comm.cc
├── caller.proto
├── k8s.yaml
├── start_service.sh
└── ...
callee
├── ...
└── ...
callee2
├── ...
└── ...
main
├── ...
└── ...
do_deploy.sh
gen_containers.sh

```

Figure 3: The unified DSL generates all deployment files automatically.

shown in Figure 3. Using the generated files, we can deploy the application to a Kubernetes environment, as illustrated in Figure 4.

III. ACCELERATING NETWORK STACK PROCESSORS USING NETBLOCKS

To accelerate network stack processors in microservice applications generated by the unified DSL, we integrate NetBlocks into the unified DSL. Our baseline custom protocol implementation is the one found in [12], and we extend it for Kubernetes environments. However, the original NetBlocks implementation does not support protocol formats other than user-defined custom ones, so it cannot handle communication frames and packets for managing Kubernetes environments properly. Because Kubernetes sends ARP frames to resolve a pod’s MAC address, DNS packets to resolve a pod’s IP address and ICMP packets to confirm connectivity, the system was crashed whenever the protocol analyzer generated by NetBlocks receives these frames and packets.

To ignore these frames and packets on the NetBlocks runtime, we extend the custom protocol with a

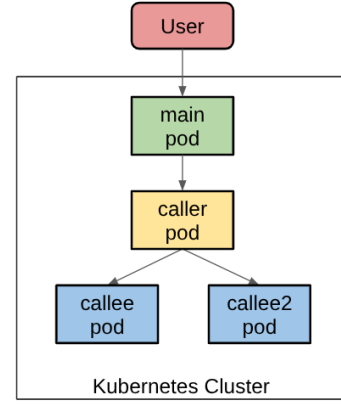


Figure 4: Resulting microservice application running on Kubernetes.

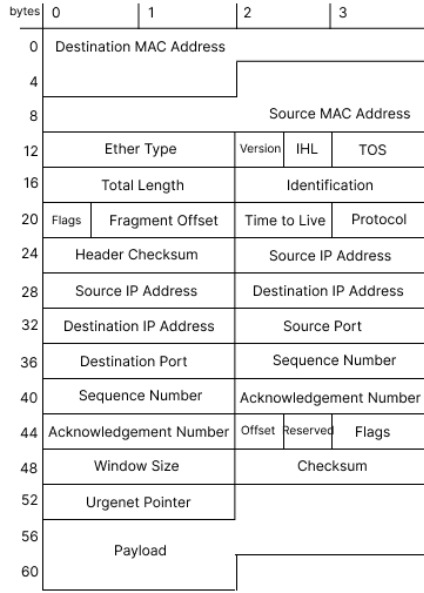
protocol_identifier field, which is similar to Ether-Type in Ethernet frames, and enable a packet filter that inspects this identifier, processing only NetBlocks packets while ignoring other types of packets. Furthermore, since NetBlocks custom protocol packets communicate with other pods through the Linux raw socket, we use Headless Service [13] to return individual pod IP addresses directly rather than a virtual ClusterIP, allowing custom protocol packets to reach the target pod without NAT. These modifications enable NetBlocks-based microservices to coexist with Kubernetes network infrastructure.

NetBlocks generates protocol stack processors from C++ source code. To integrate these into our system, we implement CMake build configurations that combine these generated protocol stack processors with application code, producing containerized binaries for Kubernetes deployment. In the unified DSL’s code generation pipeline, it extracts metadata from decorators, instantiates templates to generate files including Dockerfiles, Kubernetes manifests, NetBlocks-based communication code, build and deploy scripts, and application logic. This automated pipeline generates complete, deployable microservice applications from a single input file describing only the application logic.

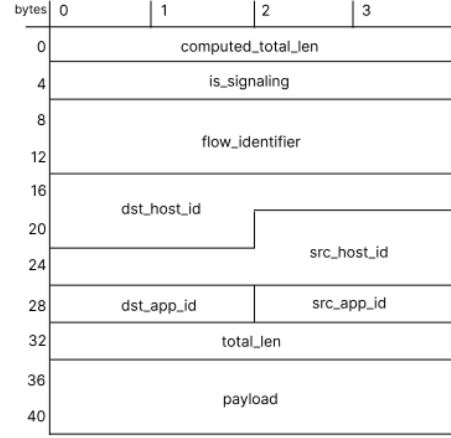
IV. EVALUATIONS

To evaluate custom protocol stacks and the unified DSL’s performance improvements, we deploy two test applications into Kubernetes environments: the Ping-Pong Application and the Prime Counter Application. Here, we conduct all evaluations on a single-node Kubernetes cluster using Minikube v1.36.0 on Arch Linux x86_64 with Kubernetes v1.33.1.

For Ping-Pong Application, we deploy a client pod and a server pod to exchange 256-byte messages and measure their round-trip latency over 100K messages. In this experiment, we evaluate four protocol implementations:



(a) Conventional TCP: Ethernet + IP + TCP



(b) NetBlocks-generated UDP-OE: Ethernet + UDP

Figure 5: Packet structures of conventional and proposed protocols

- **gRPC**: Conventional gRPC implementation written in Go, which represents the baseline communication protocol used in most microservice applications.
- **TCP**: A conventional TCP socket implementation in C.
- **TCP-NC**: A custom protocol generated using NetBlocks, where the TCP-like features, except for congestion control features, are implemented on the layer of Ethernet.
- **UDP-OE**: A custom protocol generated using NetBlocks, where the UDP-like features are implemented on the layer of Ethernet.

Figure 5 shows the packet structures of conventional TCP and the NetBlocks-generated UDP-OE protocol. We observe that the UDP-OE reduces the protocol header size compared to the conventional TCP. The UDP-OE eliminates the IP layer and just implement a simplified UDP-based transport on the layer of Ethernet, it can reduce the header size from 62 bytes of the original TCP to 44 bytes. More importantly, as demonstrated in [5], NetBlocks can achieve over an order-of-magnitude latency reduction by optimizing protocol stack processors and bypassing conventional layer-by-layer processing. This protocol stack optimization is expected to contribute to performance improvements more than just the modest reductions in header size.

Figure 6 shows the cumulative distribution function (CDF) plot of round-trip latency over 100,000 requests in the Ping-Pong Application. The median latencies are: gRPC 82 μ s, TCP 9 μ s, TCP-NC 5 μ s, and UDP-OE 2 μ s. By replacing gRPC with the NetBlocks-generated UDP-OE protocol, we achieve approximately 41 times speedup in the communication layer. From the results, we observe that the protocol stack optimization performed by NetBlocks is highly effective and delivers significant performance improvements. Also, this re-

Table 1: Lines of code comparison between the unified DSL input and generated output for a tiny microservice application

Component	Input	Output
Application logic	42	66
Dockerfiles	0	135
Kubernetes manifests	0	260
Communication code	0	1,629
Build files	0	490
Deployment scripts	0	398
Total	42	2,978

sult implies that the overhead of HTTP/2 in gRPC occupies more than that of TCP/IP.

For the Prime Counter Application, we evaluate the end-to-end latency of a microservice-based application shown in Figure 7. The application consists of four microservices shown in Figure 8, including `main` for starting microservice applications, `calc` for investigating the maximum prime number less than or equal to the given input, `is_prime` for checking whether it is a prime number, and `do_sum` for calculating a sum of sequence from 0 to a given prime number. All inter-service communication transfers a single `int32` or `bool` data. Here, the number of inter-service communications depends on the input value. If we assume $a = 100,000$ as the input, the number of inter-service communications becomes 12 as shown in Figure 8. We feed random integer numbers as input values. This application exercises realistic microservice characteristics, including multi-hop communication and computation at each service. The architecture represents typical microservice workloads where both communication and computation contribute to end-to-end latency.

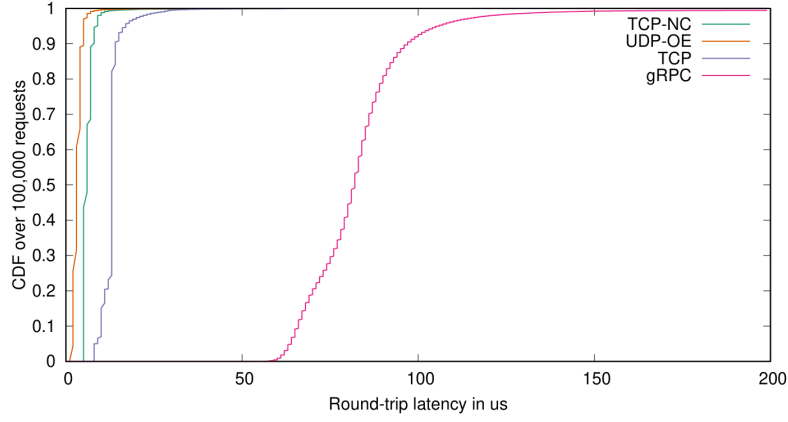


Figure 6: CDF of the round-trip latency between two pods over 100,000 messages for the Ping-pong application with the 4 protocol implementations.

```
@msvc(port="50001")
def is_prime(n: int) -> bool:
    factors = 0
    for i in range(2, n):
        if n % i == 0:
            factors += 1
    return factors == 0

@msvc(port="50002")
def do_sum(n: int) -> int:
    sum = 0
    nums = range(n)
    for k in nums:
        sum = sum + k
    return sum

@msvc(port="50003")
def calc(n: int) -> int:
    idx = 0
    for i in range(n, 0, -1):
        if is_prime(i):
            idx = i
            break
    r = do_sum(idx)
    return r

def main(n: int) -> int:
    r = calc(n)
    return r
```

Figure 7: The unified DSL code of Prime Counter Application.

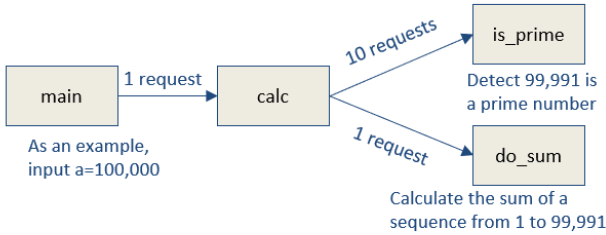


Figure 8: The microservices in the Prime Counter Application.

First, we evaluated developer productivity by comparing the lines of code (LoC) required for manual implementation versus the DSL-based development. Developers write application logic with unified DSL decorators, and it automatically generates all necessary deployment files. Table 1 shows that the input source code of the Prime Counter Application consists of only 42 lines, while the unified DSL generates 2,978 lines of microservice application files. This result demonstrates that developing microservice applications with our DSL improves developer productivity, reducing the source code which developer must write and maintain approximately 70 times.

We confirmed functional equivalence by implementing automatic verification that the output of the Prime Counter Application generated by our DSL matches that of directly executing the input Python program, and we achieved correct results in the DSL-generated microservice.

Next, we evaluate the end-to-end latency over 100,000 executions for the Prime Counter Application. Figure 9 shows the CDF of gRPC and UDP-OE. We observe that the median latency of gRPC is 39.6 ms, and that of UDP-OE is 0.12 ms. By replacing gRPC with the UDP-OE protocol, we achieve approximately 330 times speedup in end-to-end latency. We note that the average communication count among microservices in this experiment is 8.68.

While the communication layer alone achieves a 41 times speedup, the end-to-end performance improvement reaches 330 times, demonstrating that communication layer optimization significantly contributes to overall application performance. This means that the overhead of HTTP/2 in gRPC affects the End-to-end application latency more than that of simple ping-pong communication.

V. RELATED WORKS

Prior work has identified several requirements for DSLs for cloud computing environments, including infrastructure abstraction, dynamic adaptability, and declarative dependency expression. Cvetković et al. propose Dirigent, a clean-slate

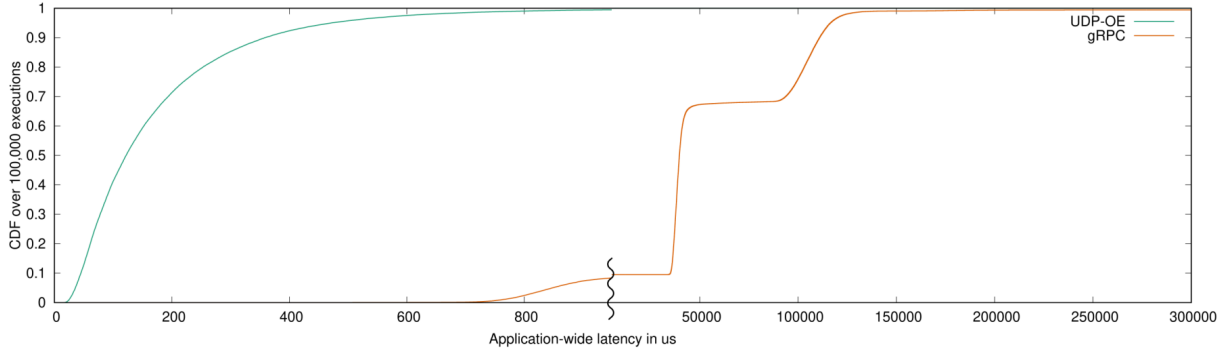


Figure 9: CDF of the end-to-end latency over 100,000 executions for the Prime Counter Application compared to gRPC between UDP-OE.

FaaS orchestration system that minimizes cluster management abstractions [6]. Mars et al. propose the Jaseci runtime and Jac programming language, which raise the level of abstraction through graph-based data representation and dynamic action binding, enabling a single developer to build and maintain production-scale AI applications [7]. Patterson et al. propose HiveMind, a hardware-software system stack for edge swarm coordination that provides a declarative task graph DSL with explicit constraint specification and an FPGA-based acceleration fabric to mitigate network communication overheads [8].

Firestone et al. demonstrate low-latency packet processing through FPGA offloading on Azure SmartNICs [14], targeting large-scale cloud datacenter deployments. In contrast, our approach targets both on-premise and cloud environments deployed by individuals or organizations. Inspired by this work, we plan to implement NetBlocks-generated protocol stack processors on FPGA-based SmartNICs to achieve further latency reduction through kernel bypass.

VI. CONCLUSION

In this paper, we presented an approach to accelerate microservice communication by integrating NetBlocks-generated custom protocol stacks into the unified DSL. Our evaluation demonstrates two key results: (1) the unified DSL improves developer productivity 70 times. (2) NetBlocks-generated UDP-OE protocol stack achieves 330 times end-to-end performance improvement compared to gRPC in a tiny application. These results substantiate that the unified DSL addresses the complexity of microservice development and the communication bottleneck in microservice applications.

ACKNOWLEDGMENT

A part of this work was based on results obtained from the project, “Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems” (JPNP20017), commissioned by the New Energy and Industrial Technology Development Organization (NEDO), and Japan Science and Technology Agency (JST) as part of Adopting Sustainable Partnerships for Innovative Research Ecosystem (ASPIRE), Grant Number JPMJAP2430.

REFERENCES

- [1] A. Zaalouk, A. Jones, A. Velichkevich, B. Kurktchiev, C. Chin, C. Zhang, C. Misale, H. Chen, J. Roberts, K.-H. Chen, M. Bhandaru, M. Yao, N. Raghunath, P. Pan, R. Kakodkar, R. Pandey, R. Aravena, R. Petty, R. Taylor, S. Sheikh, S. Wilson, T. Thorley, and V. Lu, “Cnfc cloud native ai white paper,” tech. rep., Cloud Native Computing Foundation, 2024. Accessed 2025-12-05.
- [2] Cloud Native Computing Foundation, “Cnfc annual survey 2023,” <https://www.cncf.io/reports/cnfc-annual-survey-2023/>, 2024. Accessed 2025-12-04.
- [3] gRPC Authors, “grpc,” <https://grpc.io/>, 2025. Accessed 2025-12-05.
- [4] I. Nakai, T. Fukai, T. Hirofuchi, and Y. Sato, “Comprehensive performance evaluation of microservices on confidential containers in multi-access edge computing environments,” in *2025 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 94–104, 2025.
- [5] A. Brahmakshatriya, C. Rinard, M. Ghobadi, and S. Amarasinghe, “Netblocks: Staging layouts for high-performance custom host network stacks,” *Proc. ACM Program. Lang.*, vol. 8, June 2024.
- [6] L. Cvetković, F. Costa, M. Djokic, M. Friedman, and A. Klimovic, “Dirigent: Lightweight serverless orchestration,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP ’24*, p. 369–384, ACM, Nov. 2024.
- [7] J. Mars, Y. Kang, R. Daynauth, B. Li, A. Mahendra, K. Flautner, and L. Tang, “The jaseci programming paradigm and runtime stack: Building scale-out production applications easy and fast,” *IEEE Computer Architecture Letters*, vol. 22, no. 2, pp. 101–104, 2023.
- [8] L. Patterson, D. Pigorovsky, B. Dempsey, N. Lazarev, A. Shah, C. Steinhoff, A. Bruno, J. Hu, and C. Delimitrou, “Hivemind: a hardware-software system stack for serverless edge swarms,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA ’22*, (New York, NY, USA), p. 800–816, Association for Computing Machinery, 2022.
- [9] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [10] A. Brahmakshatriya and S. Amarasinghe, “Buildit: a type-based multi-stage programming framework for code generation in c++,” in *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization, CGO ’21*, p. 39–51, IEEE Press, 2021.
- [11] Y. Futamura, “Partial evaluation of computation process—an approach to a compiler-compiler,” *Higher-Order and Symbolic Computation*, vol. 12, pp. 381–391, December 1999. Revised version of the 1971 paper.
- [12] BuildIt-lang, “Netblocks pldi 2024 artifacts,” <https://github.com/BuildIt-lang/net-blocks-pldi24-artifacts>, 2024. Accessed: 2024-12-24.
- [13] The Kubernetes Authors, “Service: Headless services,” <https://kubernetes.io/docs/concepts/services-networking/service/#headless-services>, 2025. Accessed: 2026-03-21.
- [14] Firestone et al., “Azure accelerated networking: Smartnics in the public cloud,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pp. 51–66, 2018.